

Révision

Notions de cours :

1-Citer deux avantages du framework Spring boot :

-Centralisation de la configuration dans un seul fichier **application.properties**.

-Création des applications autonomes (avec **serveur web embarqué (Apache Tomcat)**)

-La gestion des dépendances est simplifiée grâce aux **starters** (Ensemble de dépendances qui incluent les bibliothèques et les configurations dont on aura besoin)

2-Quelles sont les principales dépendances à ajouter lors de la création d'un projet Spring Boot ? Citez-les et expliquez brièvement le rôle de chacune.

Spring web (Sert à créer des applications web intégrant automatiquement un serveur web : Tomcat par défaut) / **Spring Data JPA** (Simplifie l'accès à la BD) / **MySQL driver** (Pont entre Java et MySQL) / **Lombok** (Gestion des getters, setters, constructeurs...)

3-Quel est le rôle de Maven dans un projet Spring Boot ?

Maven est un outil de gestion de projets JAVA

- ➔ Facilite la création d'un projet JAVA (Structure de projet standardisée, **arborescence prédéfinie**)
- ➔ Facilite la gestion d'un projet JAVA (**Cycle de vie uniforme / gestion centralisée via le fichier pom.xml**)
- ➔ Facilite la gestion des dépendances (Résolution automatique des bibliothèques -> téléchargement depuis les repositories (MAVEN Central))

4- Citer, dans l'ordre, les principales étapes que suit Maven lors de la construction d'un projet ? Expliquez brièvement l'utilité de chacune

validate (vérifie que le projet est correctement configuré)

compile (Compile le code source JAVA en .class)

test (exécute les tests unitaires)

package (création du livrable dans le dossier target)

verify (vérifier la qualité du livrable)

install (installer le livrable dans le repository local Maven (.m2/repository))

deploy (publier le livrable dans un repository distant, exemple : Nexus)

5-Qu'est-ce qu'un **ORM** ? Quel est son rôle ? et quel est l'ORM utilisé par Spring Boot ?

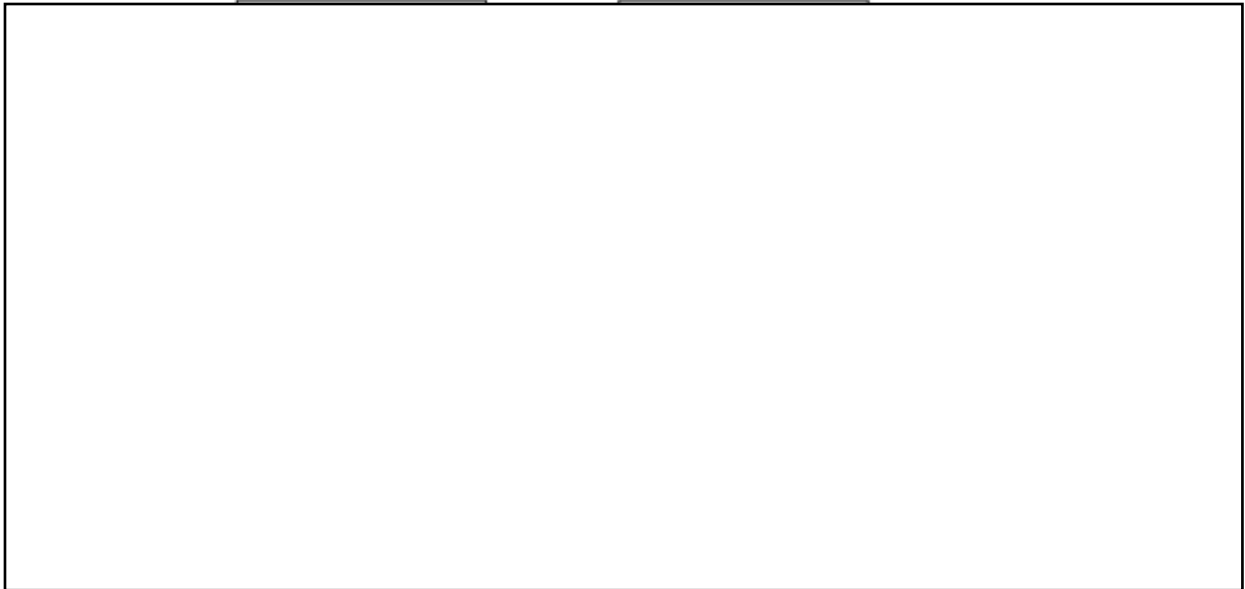
ORM(Object Relational Mapping) , c'est un outil permettant de faire le **mapping/la correspondance** entre les **tables d'une BD relationnelle** et les **objets** d'un langage de programmation.

L'ORM le plus connu dans le monde JAVA , et qui est utilisé par Spring Boot , c'est **Hibernate**.

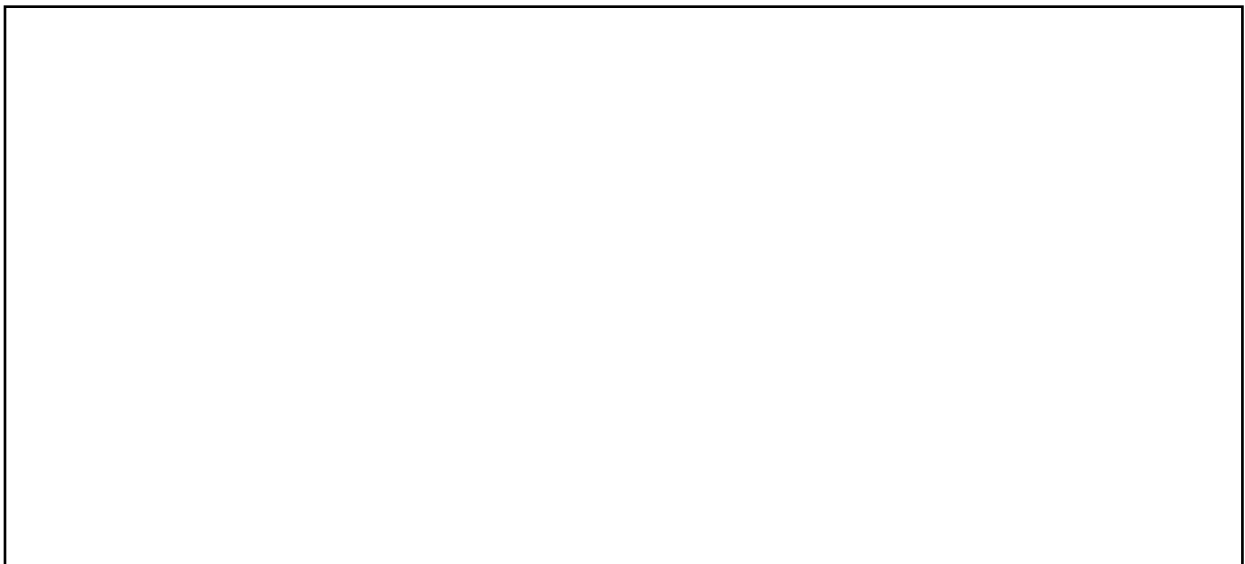
6-Quelle est la différence entre **architecture logique** et **architecture physique** ?

L'architecture logique décrit l'organisation interne du logiciel en couches et la manière dont elles interagissent, tandis que **l'architecture physique** décrit l'infrastructure sur laquelle le logiciel s'exécute.

7-Schématisez l'**architecture physique** d'une application **n-tiers (n = 4)**
(Framework back-end : Spring Boot / Framework front-end : Angular).

A large, empty rectangular box with a black border, intended for the student to draw a diagram of the physical architecture of an n-tier application. The box is currently blank.

8-Schématisez l'**architecture logique** d'une application Spring Boot en précisant les différentes couches.

A large, empty rectangular box with a black border, intended for the student to draw a diagram of the logical architecture of a Spring Boot application, showing the different layers. The box is currently blank.

9-Définir l'injection de dépendances. Quel est son rôle et pour quel problème constitue-t-elle une solution efficace ?

Dans un programme classique, c'est le développeur qui crée les instances directement dans le code (exemple : dans la classe A : `B b = new B();`). Avec l'injection de dépendances, qui est une forme d'**IoC** (Inversion of Control), Spring prend en charge la création des instances et les injecte automatiquement dans les classes qui en ont besoin – au lieu que la classe A instancie sa dépendance B, c'est Spring qui injecte une instance de B dans A.

Cette approche résout efficacement le problème du **couplage fort** entre les composants et facilite la maintenance.

10-Citer les différentes façons de réaliser l'injection de dépendances en Spring.

- ➔ @Autowired
- ➔ Par constructeur
- ➔ Par mutateur (les setters)

11-Qu'est-ce qu'un DTO ?

DTO est l'abréviation de **Data Transfer Object**. Son rôle est de **transporter les données entre les différentes couches** d'une application ou entre le **backend et le frontend**, sans exposer directement les entités du domaine.

11.1-Citer deux avantages de l'utilisation des DTO.

Optimisation (On envoie seulement ce dont le client a besoin , et non toute l'entité).

Sécurité (On ne veut pas exposer toutes les informations de l'entité (par exemple le mdp peut être masqué)

Peuvent aider à **éviter les boucles infinies**.

12-Quelle est la couche responsable de la communication avec la base de données ?

Citer les types d'interfaces utilisées et préciser les relations d'héritage entre elles.

C'est la **couche Repository** qui est responsable de la communication avec la BD. Types d'interfaces utilisées :

CRUD Repository



Paging & Sorting Repository



JPA Repository

13-Pour développer des méthodes personnalisées, on peut utiliser les **keywords**, **JPQL** et **Native Query**.

Expliquer la différence entre ces approches.

Keywords (ce sont des méthodes basées sur le **nommage**. Elles permettent de générer automatiquement des requêtes à partir du nom de la méthode, à condition d'avoir un accès direct aux attributs des entités.)

JPQL(Java Persistence Query Language) : ce sont des requetes personnalisés orientées JAVA , on manipule **les entités et leurs attributs coté code**, sans se soucier des tables physiques)

Native Query(ce sont des requêtes SQL native, là ou on manipule directement **les tables ainsi que les colonnes coté BD**)

14-Quel est le rôle du **Scheduler** dans Spring Boot ?

Le **Scheduler** fait partie du module Spring Framework. C'est une fonctionnalité intégrée qui permet de **planifier automatiquement l'exécution de tâches** dans l'application, sans intervention manuelle.

15-Quel est le rôle de **Spring AOP** ?

L'AOP est un paradigme de programmation qui permet de **séparer le code technique**(tout ce qui est transversale , exemple : performance , journalisation, sécurité ..) **du code métier**(la vraie logique de l'application) dans une application spring boot.

15.1- Sur quels **principes** se base l'AOP ? Citer le problème résolu par chaque principe.

L'AOP se base sur les principes :

SOC (Separation Of Concerns) : c'est une solution au problème de mélange de code (**Tangling**)

DRY (Dont Repeat Your Self) : c'est une solution au problème de répétition du même code technique (**scattering**)

15.2- Citer les **types d'advice** en Spring AOP et expliquez la différence entre eux.

@Before : exécuté **avant** la méthode métier.

@After : exécuté **après** l'exécution de la méthode métier, que celle-ci réussisse ou non.

@AfterReturning : exécuté **après l'exécution réussie** de la méthode métier (aucune exception n'a été levée).

@AfterThrowing : exécuté **lorsqu'une exception est levée** pendant l'exécution de la méthode métier.

@Around : exécuté **autour de la méthode**, c'est-à-dire **avant et après** son exécution.

16-Expliquer les **PointCut** suivants :

- `@Before("execution(* tn.esprit.spring.service.*(..))")`

Avant l'exécution de toutes les méthodes de toutes les classes dans le package service

N'importe quel type de retour

Toutes les classes du package service

Toutes le méthodes

Peut importe les paramètres

- `@After("execution( void tn.esprit.spring.service.add*(..))")`

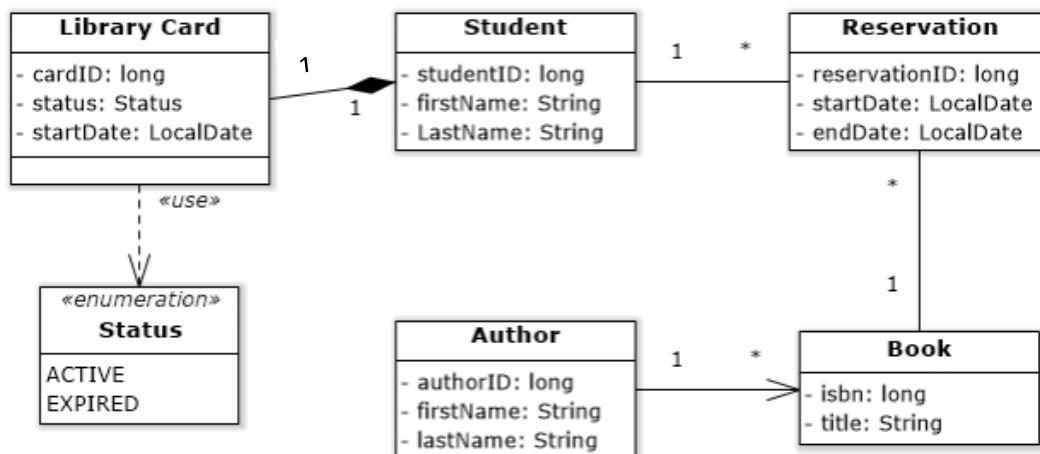
Après l'exécution de toutes les méthodes du package tn.esprit.spring.service commençant par add et ayant comme type de retour void, peu importe leurs paramètres.

- `@Before("execution(* set*(..))")`

Avant l'exécution de toutes les méthodes dont le nom commence par set, dans toute l'application, quel que soit leur type de retour ou leurs paramètres.

Etude de cas

On vous propose d'implémenter une application simplifiée de gestion de bibliothèque suivant le diagramme de classes ci-dessous :



Entités et Associations :

Complétez les entités ci-dessous en ajoutant les annotations nécessaires ainsi que les relations manquantes afin de construire le **schéma de la base de données** conformément au diagramme de classes fourni ci-dessus, en respectant les contraintes suivantes :

- Chaque entité doit être mappée en tant que table au niveau de la base de données(**@TODO1**)
- Pour chaque entité, les **getters(@TODO2)**, les **setters(@TODO3)**, ainsi qu'un **constructeur paramétré(@TODO4)** et un **constructeur non paramétré(@TODO5)** doivent être générés **automatiquement**.
- Les identifiants des entités doivent être gérés selon la stratégie **d'auto-incrément(@TODO6)**.
- L'énumération doit être enregistrée sous forme de **chaîne de caractères(@TODO7)** dans la base de données.
- Pour l'entité **Student**, les attributs **firstName** et **lastName** doivent être non nuls(**@TODO8**).
- Pour l'entité **Book**, l'attribut **title** doit être unique(**@TODO9**).
- Pour éviter les boucles infinies lors de la sérialisation des étudiants, la liste des réservations de chaque étudiant ne doit pas être sérialisée.(**@TODO10**)

1-Idéalement combien de tables doivent être généré au niveau de la base de données ?

Idéalement, **6 tables** seront générées côté base de données :

- Les **5 premières tables** correspondent aux entités : LibraryCard, Student, Author, Book et Reservation.
- La **6^{ème} table** est une **table de jointure**, créée suite à la relation **OneToMany unidirectionnelle** entre Author et Book.

<pre> (@TODO1) @Entity (@TODO2) @Getter (@TODO3) @Setter (@TODO4) @AllArgsConstructor (@TODO5) @NoArgsConstructor public class LibrabyCard { @Id (@TODO6) @GeneratedValue(strategy = GenerationType.IDENTITY) Long cardID; (@TODO7) @Enumerated(EnumType.STRING) Status status; LocalDate startDate; // (Relation avec Student) @OneToOne(mappedBy = "libraryCard") Student student ; } </pre>	<pre> (@TODO1) @Entity (@TODO2) @Getter (@TODO3) @Setter (@TODO4) @AllArgsConstructor (@TODO5) @NoArgsConstructor public class Student { @Id (@TODO6) @GeneratedValue(strategy = GenerationType.IDENTITY) Long studentID; (@TODO8) @NotNull String firstName; (@TODO8) @NotNull String lastName; //(Relation avec LibraryCard) @OneToOne(cascade = CascadeType.ALL) LibraryCard libraryCard ; (@TODO10) @JsonIgnore //(Relation avec Reservation) @OneToMany(mappedBy = "student") List<Reservation> réservations ; } </pre>	<pre> (@TODO1) @Entity (@TODO2) @Getter (@TODO3) @Setter (@TODO4) @AllArgsConstructor (@TODO5) @NoArgsConstructor public class Reservation { @Id (@TODO6) @GeneratedValue(strategy = GenerationType.IDENTITY) Long reservationID; LocalDate startDate; LocalDate endDate; //(Relation avec Student) @ManyToOne Student student ; //(Relation avec Book) @ManyToOne Book book ; } </pre>
---	--	---

<pre>public enum Status { ACTIVE, EXPIRED }</pre>	<pre>(@TODO1) @Entity (@TODO2) @Getter (@TODO3) @Setter (@TODO4) @AllArgsConstructor (@TODO5) @NoArgsConstructor public class Book { @Id (@TODO6) @GeneratedValue(strategy = GenerationType.IDENTITY) Long isbn; (@TODO9) @Column(unique = true) String title; // (Relation avec Reservation) @OneToMany(mappedBy = "book") List<Reservation> réservations ; // (Relation avec Author) Pas de visibilité (OneToMany unidirectionnelle) }</pre>	<pre>(@TODO1) @Entity (@TODO2) @Getter (@TODO3) @Setter (@TODO4) @AllArgsConstructor (@TODO5) @NoArgsConstructor public class Author { @Id (@TODO6) @GeneratedValue(strategy = GenerationType.IDENTITY) Long authorID; String firstName; String lastName; // (Relation avec Book) @OneToMany List<Book> books ; }</pre>
---	--	--

Repositories/Services/Controllers

1-Complétez le repository ci-dessous afin qu'il hérite de l'interface CrudRepository pour l'entité Student.

```
public interface StudentRepository extends CrudRepository <Student, Long> {

}
```

2-Complétez le service ci-dessous et implémentez un contrôleur permettant d'ajouter un nouveau étudiant(Student) ainsi que sa LibraryCard associée.

```
@Service
public class IStudentServiceImp implements IStudentService{

    (injectez le repository correspondant en utilisant l'annotation @Autowired)
    @Autowired

    Private StudentRepository studentRepository ;
    @Override
    public Student addStudent (Student student) {

    Return studentRepository .save(student) ;

}
```


Remarque : Lors des tests des endpoints avec un outil tel que **POSTMAN**, la méthode permettant d'ajouter un nouveau Student sera accessible via l'URL suivante :

http://localhost:8080/student/addStudent

```
@RestController
@RequestMapping("/student")
@AllArgsConstructor
public class StudentController {

Injectez le service concerné en utilisant l'injection par constructeur.

private final IStudentService studentService ;

@PostMapping("/addStudent")
public Student addStudent(@RequestBody Student student){

Return studentService. addStudent(student) ;
}
```

Affectation

Rappel :

- Pour une **relation unidirectionnelle**, l'affectation se fait à partir de l'entité **porteuse de la relation**, c'est-à-dire celle **d'où part la flèche** dans le diagramme de classes.
- Pour une **relation bidirectionnelle**, l'affectation se fait au niveau de l'entité **propriétaire de la relation** (parent)

(L'affectation et la désaffectation des relations se font à l'aide des méthodes **set** appropriées.)

(Exemple 1: OneToOne)

Complétez le service ci-dessous et exposer la méthode dans un contrôleur permettant d'ajouter un LibraryCard et l'affecter à un Student déjà existant dans la BD .

```
@Override
public Student affecterLibraryCardAstudent (LibraryCard c, Long studentId) {

    //Chercher student par son id

    Student s = studentRepository.findById(studentId). orElseThrow(() -> new RuntimeException("student
not found"));

    //Enregistrer LibraryCard

    LibraryCard libraryC = libraryCardRepository.save(c) ;

    //Affecter LibraryCard à Student

    s.setLibraryCard(libraryC) ;

    //Sauvegarder le nouveau état de l'objet student

    Return studentRepository.save(s) ;

}
```

```
@PutMapping("/affecterCardAstudent/{ studentId }")

public Student affecterLibraryCardAstudent (@RequestBody LibraryCard c,@PathVariable Long
studentId) {

    return studentService. affecterLibraryCardAstudent(c,studentId) ;

}
```

(Exemple 2 : OneToMany)

Complétez le service ci-dessous permettant d'ajouter un student avec les réservations associées.

Il suffira donc par la suite de sauvegarder un objet student contenant une liste de réservations et affecter l'étudiant (child) aux réservations (Parent).

Remarque : Comme l'ajout des réservations se fait à partir de l'entité **Student**, il est nécessaire d'utiliser une cascade de type **ALL** sur la relation avec **Reservation**, comme suit :

```
@OneToMany(mappedBy = "student", cascade = CascadeType.ALL)
List<Reservation> reservations;
```

```

@Override
Public Student addStudentEtReservationsAssociées(Student s){
//Sauvegarder l'objet fils student
Student studentSaved = studentRepository.save(s); (grâce à la cascade, les réservations seront sauvegardées automatiquement)
//parcourir la liste des réservations (parent)
if (s.getReservations() != null) {
    for (Reservation r : s.getReservations()) {
//Affecter le child student au parent réservation
r.setStudent(studentSaved);
//Sauvegarder le parent(la réservation)
reservationRepository.save(r);
    }
Return student ;
}

```

Important : Veuillez consulter le cours sur (Spring Data Jpa-Affectation) pour voir les différents exemples d'affectation restants.

Keywords/JPQL/Native Query

1- Créer un service permettant de lister les Student selon le status de la LibraryCard en utilisant trois méthodes différentes :

- Keywords
- JPQL
- Native Query

```
public interface StudentRepository extends JpaRepository<Student, Long> {  
    //Keywords  
  
    List<Student> findByLibraryCardStatus(Status status) ;  
  
    //JPQL  
  
    @Query(" SELECT s FROM Student s JOIN s.libraryCard c WHERE c.status = :status ")  
    List<Student> findStudentsByCardStatusJPQL(@Param("status") Status status) ;  
  
    //Native Query  
  
    @Query(value = " SELECT s.* FROM student s JOIN library_card c ON l.libraryCardID =  
        c.libraryCard_ID WHERE c.status = :status" , nativeQuery = true)  
    List <Student> findStudentsByCardStatusNative(@Param("status") String status) ;  
}
```

Scheduler :

Remarque : Pour activer le mécanisme de planification (scheduler) dans Spring Boot , il faut ajouter l'annotation **@EnableScheduling** au-dessus de la classe main du projet.

1- En utilisant Spring Scheduler, proposez une méthode qui se déclenche **30 secondes après la fin de l'exécution précédente** et qui met à jour le status des LibraryCards de ACTIVE à EXPIRED, uniquement si leur date de fin est passée (inférieure à la date actuelle).

```
@Scheduled(fixedDelay = 30, timeUnit = TimeUnit.SECONDS)

public void updateStatusLibraryCard() {

    // Récupérer la liste des LibraryCards ayant le statut ACTIVE

    // (méthode à ajouter au niveau du repository LibraryCard afin de filtrer les cartes selon leur statut)

    List<LibraryCard> activeCards = libraryCardRepository.findByStatus(LibraryCardStatus.ACTIVE);

    //Parcourir la liste des cartes et changer leurs status vers EXPIRED (si endDate inférieure à la date
    actuelle)

    for (LibraryCard card : activeCards) {

        if (card.getEndDate().isBefore(LocalDate.now())) {

            card.setStatus(LibraryCardStatus.EXPIRED);

        }

    }

    //Enregistrer les cartes

    libraryCardRepository.saveAll(activeCards);

}
```

2-Si l'on souhaite que la méthode développée précédemment soit déclenchée **toutes les 30 secondes**, comment organiseriez-vous son exécution ?

```
@Scheduled(fixedRate = 30, timeUnit = TimeUnit.SECONDS)
```

3-Si l'on souhaite que la méthode soit déclenchée **tous les jours à 08h30**, comment organiseriez-vous son exécution pour respecter ce planning ?

```
@Scheduled(cron = "0 30 8 * * *")
```

AOP :

1-Créer un aspect permettant

- 1.1 / d'afficher le message « Bienvenue à l'un des services de l'application : NOM_DU_SERVICE » **avant** l'exécution de chaque méthode du package services.
- 1.2/ d'afficher le message « Ajouté avec succès » **à la fin** de l'exécution **réussie** de chaque méthode du package services et qui commence par add avec un type de retour void.

```
@Aspect
@Component
@Slf4j

public class ConfigAop {

    //1.1
    @Before ("execution(* tn.esprit.examen.services.*(..) )")
    public void EntreeMethode (JoinPoint jp) {
        String name = jp.getSignature().getName();
        log.info("Bienvenue à l'un des services de l'application : " + name );
    }

    //1.2

    @AfterReturning ("execution(void tn.esprit.examen.services.add*(..))")
    public void afterAddMethods() {
        log.info("Ajouté avec succès");
    }

}
```